

GZ CURVE Automation

Project Members -

Aadi Gahrotra - NA24B021

Gunin Midha - NA24B035

Vedant Soni - NA24B020

Harsh Solanki - NA24B036

Rudrakshi Bhadoo - NA24B016

Everyone was involved in ideating and coding this project.

This project consists of three steps-

- Half Breadth Plan Using the Offset table with Tchebycheff stations
- Drawing Tchebycheff body plan from the offsets
- Krylov's method of calculating GZ

1. Waterlines Creation

Each waterline is fitted using a 7th-degree polynomial with

`numpy.polyfit(x, y, 7, w = weights)`.

This creates a smooth representation of each waterline.

`numpy.poly1d(coefficients)` converts the coefficients into a usable polynomial function for evaluation and plotting.

Adding Stem & Stern Boundary Conditions

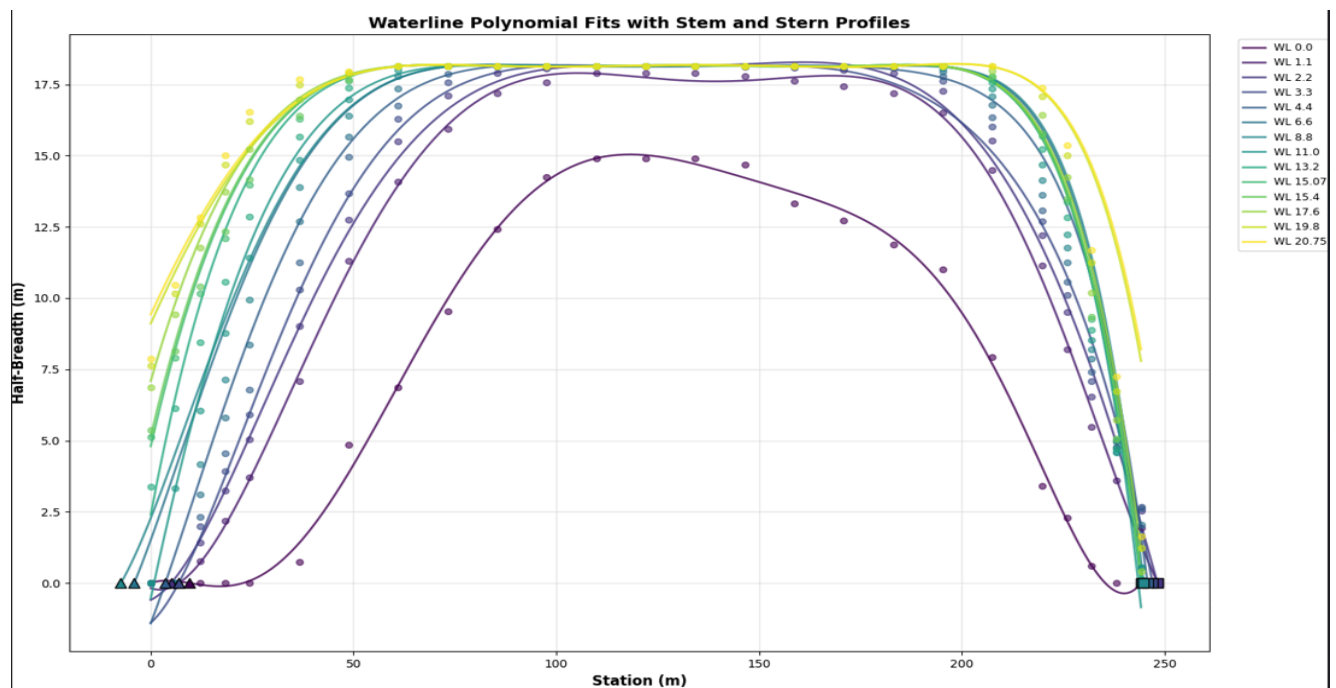
- If the waterline appears in predefined lists, the script adds a stern point ($x = \text{stern offset}$) or a stem point ($x = 244.2 + \text{stem offset}$).
 - These added points force the polynomial to pass through the physical hull endpoints (half-breadth = 0).
-

Weighting Conditions (Important for Curve Shape)

- Normal points, weight = 1
- Stern and Stem point, weight = **500**
- Any point with half-breadth near **18.15 ± 0.01** , weight = **100**
- These weights strongly influence polyfit so the curve respects key geometry.

Curve Evaluation for Plotting

- A smooth x-range is generated using `np.linspace()` and evaluated using the `poly(x)` object to produce a smooth curve.



2. Tchebycheff Offsets

Loads fitted waterline polynomials

Loads `waterline_equations.json` and extracts the polynomial coefficients for all waterlines including WL **15.07**, which is used as the reference LWL waterline.

Evaluates polynomials manually

Uses a custom function **eval_poly(coefficients, x)** to compute polynomial values (written manually instead of using `poly1d`).

Finds LWL using WL 15.07

- Uses a root-finding approach:
 - Generates a fine x-range (`np.linspace`)
 - Detects **sign changes** to locate where WL 15.07 crosses the x-axis
 - Applies **fsolve()** for precise zero-finding
 - The two roots = **forward and aft endpoints of LWL**
 - Computes **LWL** and **LWL/2** midpoint.

Defines 12 Tchebychev vertical stations

- Predefined fractions of **LWL/2** are taken $\pm[0.0669, 0.2887, 0.3667, 0.6330, 0.7113, 0.9331]$.
- Actual x-coordinates = $\text{LWL}/2 + \text{fraction} \times (\text{LWL}/2)$.

Determines valid x-range for every waterline

- For each waterline polynomial:
 - Searches for its x-axis crossings using sign change detection + `fsolve()`.
 - These define the **valid usable range** of that waterline.
 - If not found - assigns full range as fallback.

Computes intersection at each station

- For each waterline and each vertical station x:
 - Evaluates polynomials at that x using `eval_poly`.
 - Apply strict value conditions:
 - (1)** if $y < 0$ - set to **0**
 - (2)** if $18.10 < y \leq 20$ - force to **18.15**

3. Krylovs Method

This code reads the previously generated `tcchebychev_intersection_points.csv` containing half-breadths at 12 stations for all waterlines.

Spline fitting of station curves

- The function **`fit_curve()`** builds a smooth curve for each station:
 - Uses **`np.polyfit(y_below, x_below, degree)`** when ≥ 4 points are available
 - Falls back to **`poly = np.poly1d(coeffs)`** when spline cannot be fitted
 - Enforces the vertical cap at **± 18.15 m**
 - Returns a smooth (x,y) contour for each station

Rotated waterline intersection calculation

- The core function **`calculate_intersections(angle, F_x, F_y)`**:
 - Computes slope from heel angle
 - For each station and side (original/mirror), fits its spline
 - Computes intersection with the rotated waterline using minimum distance
 - If intersection distance > 0.5 - fallback rules:
 - (1)** upper fallback = horizontal line at 20.757 m(deck line)
 - (2)** lower fallback = baseline at 0 m
 - Enforces half-breadth limit at **± 18.15 m**
 - Returns intersection coordinates and distance from rotation center

Waterplane parameters for each angle

- Function **calculate_waterplane_parameters()**:
 - Splits distances into **a-values** (aft) and **b-values** (forward)
 - Computes raw sums: $\Sigma(a+b)$, $\Sigma(a^2-b^2)$, $\Sigma(a^3+b^3)$
 - Calculates:
 - $Y_f = (\Sigma(a^2-b^2)/\Sigma(a+b))/2$
 - $I_x = DL \cdot \Sigma(a^3+b^3)/3$
 - $DI = DL \cdot \Sigma(a+b) \cdot Y_f^2$
 - $I_x(\theta) = I_x - DI$
 - $BM(\theta) = I_x(\theta)/\nabla$
 - Returns hydrostatic quantities for GZ computation

Yf and Yf/2 method for new center of floatation

- Function **calculate_new_cog_pdf_method()** updates the floating center (F_x, F_y) for the next heel angle:
 - Uses $\delta\theta$ and Y_f
 - Computes shifts along previous and next waterlines
 - Projects onto new rotated axis
 - Returns updated coordinates

Angle loop performing full hydrostatic analysis

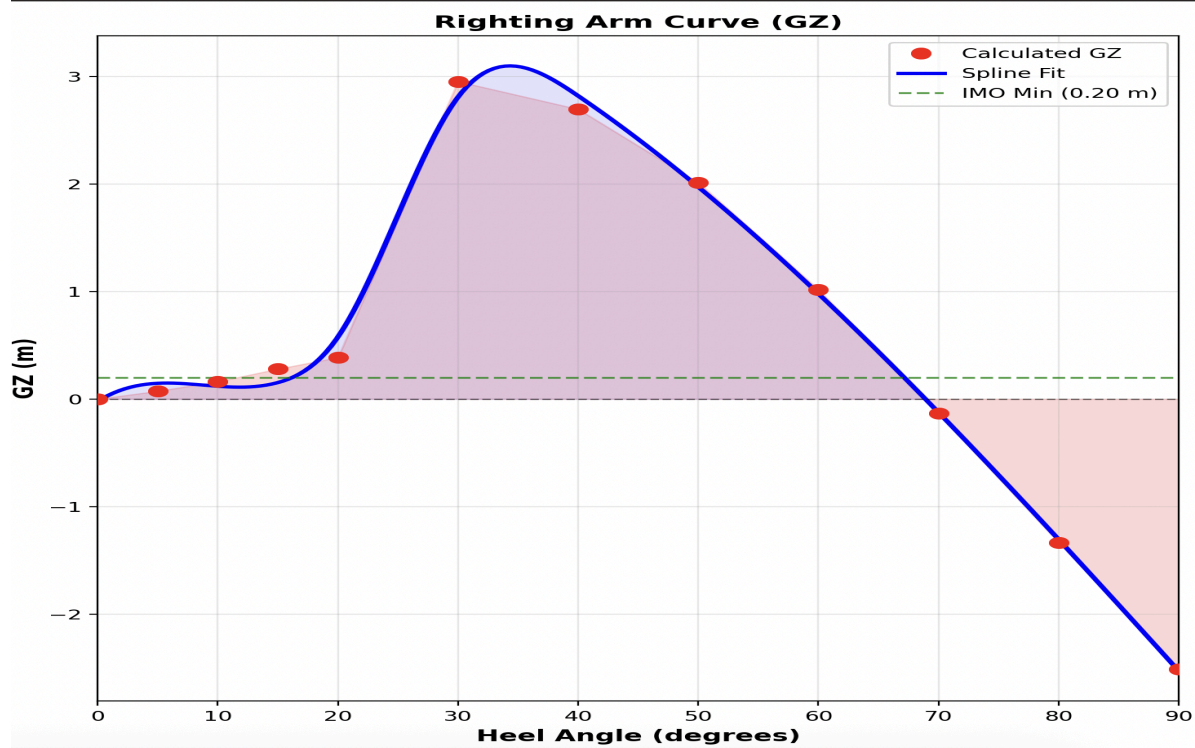
- For each heel angle (0° to 90°):
 - Finds all intersections
 - Computes waterplane parameters
 - Computes $BM(\theta)$
 - Updates rotation center for next angle

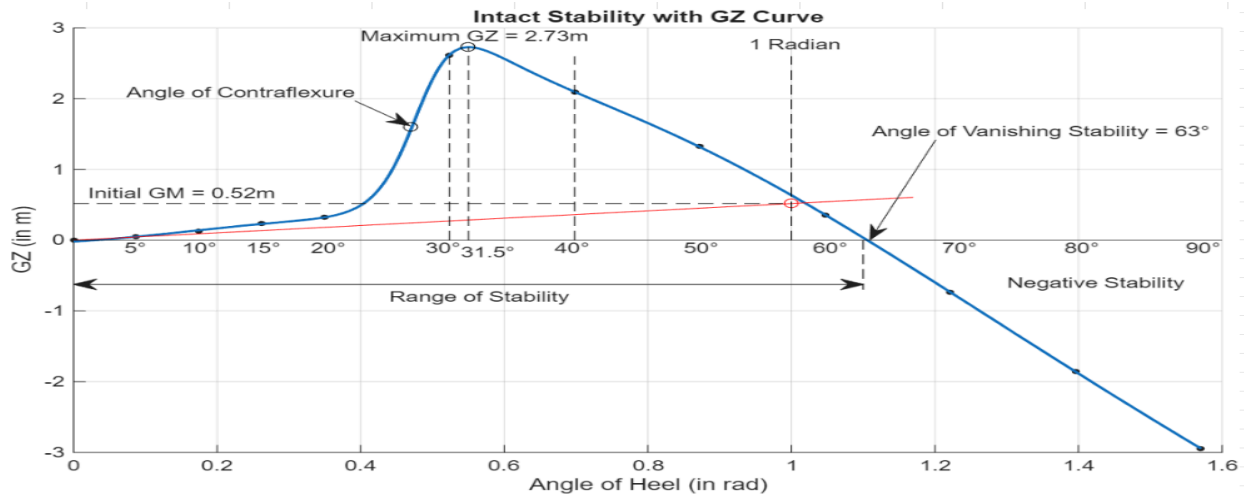
Krylov integration for KN and GZ

- Computes $BM \cdot \cos\theta$ and $BM \cdot \sin\theta$ arrays
- Integrates using Krylov method to compute $y\theta$ and $Z\theta - Z_c$
- Computes:
 - $I_c = y\theta \cdot \cos\theta + (Z\theta - Z_c) \cdot \sin\theta$
 - $KN = I_c + KB_0 \cdot \sin\theta$
 - $GZ = KN - KG \cdot \sin\theta$
- Produces final GZ curve values for all angles

Final body plan and GZ plotting

- Plots all 12 waterlines using spline
- Mirrors them for full body plan
- Fits GZ curve using UnivariateSpline and a polynomial
- Plots heel vs GZ
- This is the GZ curve obtained





This is the GZ curve found using **AUTOCAD** method

The two curves are almost similar in nature proving the fact that the code generates correct GZ curve

ALTERNATIVE WAY-

We also thought of using the offset directly to create a body plan and then using Krylov's method to find intersection and compute GM

- This way there are nearly 25 stations which are plotted on a single body plan and mirrored for calculating a and b for every auxiliary waterline
- This is a method to bypass the step of making a half breadth plan, drawing Tchebycheff stations and taking their offsets .
- The code uses the offset table and `spline = make_interp_spline(y, x, k=3)` function which draws 3 degree equations based on the given points
- The equations are plotted as well as saved in a text file which can then be used to find intersection with the auxiliary waterlines
- We were not able to implement the krylov part for this method further

